

Reliable Numerical Simulation of Financial SDEs: GBM Benchmarks and a Non-Affine Volatility Model

Project 1 (ODE IVP Extension Pathway: Financial SDE)

Team #<TEAM NUMBER>

<Name 1>, <ID 1> (Project Manager) <Name 2>, <ID 2> (Mathematical Theory)
 <Name 3>, <ID 3> (Algorithm Implementation) <Name 4>, <ID 4> (Visualization & Report)
 <Name 5>, <ID 5> (Testing & Validation)

Department of Applied Mathematics

18 September 2026

Abstract

This project develops and validates a reproducible numerical workflow for two financial stochastic differential equations (SDEs). Geometric Brownian motion (GBM) is first used as an exact benchmark: Euler–Maruyama (EM), scalar Milstein, and exact lognormal transitions are driven by identical Brownian increments. This coupling isolates time-discretisation error from sampling noise. With 300,000 paths, the fitted terminal strong-convergence slopes are 0.4899 ± 0.0026 for EM and 0.9099 ± 0.0167 for Milstein, consistent with the theoretical orders $1/2$ and 1 . The exactly evaluable bias in $\mathbb{E}[S_T]$ has fitted slope 0.9982 ± 0.0004 . At a matched mean absolute error target of 0.85 price units, EM uses 128 path steps while Milstein uses 2 under the declared step-count cost measure. The experiment also checks the terminal lognormal law and demonstrates that price-space updates are not unconditionally positivity preserving. We then solve a bounded, non-affine stochastic-volatility model in log price. Coarse paths are coupled to an $N_{\text{ref}} = 2048$ EM reference by summing fine-grid Brownian increments. The empirical strong slope is 0.5111 ± 0.0047 ; Brownian variance and correlation diagnostics pass, and all simulated log-price paths remain finite and positive. Because the reference is numerical and call-payoff bias confidence intervals include zero, no unsupported weak-order claim is made.

Keywords: stochastic differential equations; geometric Brownian motion; Euler–Maruyama; Milstein method; strong convergence; weak convergence; stochastic volatility; Monte Carlo coupling

Contents

1	Introduction and motivation	3
1.1	Problem background	3
1.2	Research questions and contribution	3
1.3	Report map	3
2	Financial SDE models	3
2.1	Geometric Brownian motion	3
2.2	Non-affine stochastic volatility	4
3	Discretisation and mathematical analysis	4
3.1	Euler–Maruyama for GBM	4
3.2	Milstein for GBM	4
3.3	Exact transition and pathwise coupling	5

3.4	Weak error and an analytic bias oracle	5
3.5	Positivity analysis	5
3.6	Correlated EM for the non-affine model	6
3.7	Nested-grid numerical reference	6
3.8	Uncertainty quantification	6
4	Implementation and reproducibility	6
4.1	Core algorithm	6
4.2	Single-file code structure	7
4.3	Reproducible protocol	7
4.4	Oracle chain and automatic checks	7
4.5	Declared cost measure	8
5	Numerical experiments and results	8
5.1	GBM strong and weak convergence	8
5.2	Matched-accuracy cost comparison	9
5.3	Terminal distribution and moment validation	9
5.4	Positivity stress test	9
5.5	Non-affine nested-grid convergence	10
5.6	Increment, finiteness, positivity, and reference checks	11
6	Limitations and robustness	12
7	Conclusions	12
	References	13
A	Reproduction record and complete console output	13
A.1	Environment and commands	13
A.2	Archived program output	14
B	Complete Python source code	14
C	AI Transparency Log	24
C.1	Tools used	24
C.2	Significant AI-assisted claim audits	24
C.3	Independent verification and decisions	24
C.4	Intellectual ownership declaration	24
D	Individual Contribution Statements	25
E	Submission evidence matrix	30

1 Introduction and motivation

1.1 Problem background

An SDE model is useful only if its numerical error can be separated from Monte Carlo uncertainty. GBM is therefore more than a toy model: its exact pathwise transition, moments, and terminal distribution provide an oracle against which an implementation can be tested. Higham’s algorithmic treatment emphasises precisely this role of coupled paths when illustrating EM and Milstein [1]; the broader convergence theory is developed by Kloeden and Platen [2] and Milstein [3].

The project asks a second, more practically relevant question: what changes when no exact transition is available? We answer it with the prescribed two-factor non-affine stochastic-volatility model. The same Brownian path is represented on nested grids, so agreement under refinement becomes evidence for convergence. It is not relabelled as an exact error. This distinction is central to reliable computational finance and underlies multilevel coupling methods [4].

1.2 Research questions and contribution

The numerical investigation addresses four questions.

1. Do coupled simulations recover the predicted GBM strong orders $1/2$ (EM) and 1 (Milstein), and weak order 1 for the mean?
2. At a matched strong-error target, which method uses fewer time-step coefficient evaluations?
3. When can price-space updates lose positivity, and does the simulated terminal law agree with the exact lognormal distribution?
4. For the non-affine model, do nested-grid EM solutions converge, pass Brownian-increment checks, remain finite and positive, and use a sufficiently resolved numerical reference?

The main contribution is a single, auditable script that implements every scheme, fixes all seeds, regenerates every figure and printed number, and runs explicit PASS/FAIL checks. The analysis deliberately separates theorem, measurement, and limitation.

1.3 Report map

Section 2 defines both models. Section 3 derives the numerical schemes and convergence estimators. Section 4 documents the implementation and validation design. Section 5 presents GBM and non-affine experiments. Section 6 identifies what the evidence does not establish, and Section 7 concludes. The complete source and AI/individual-contribution records appear in the appendices.

2 Financial SDE models

2.1 Geometric Brownian motion

Let $(W_t)_{t \geq 0}$ be a standard Brownian motion. Under the physical measure, GBM is

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad S_0 > 0. \quad (1)$$

For $f(s) = \log s$, Itô’s formula gives

$$\begin{aligned} d \log S_t &= f'(S_t) dS_t + \frac{1}{2} f''(S_t) (dS_t)^2 \\ &= \left(\mu - \frac{1}{2} \sigma^2 \right) dt + \sigma dW_t. \end{aligned} \quad (2)$$

Integrating (2) yields the exact solution

$$S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma W_t\right]. \quad (3)$$

Consequently,

$$\mathbb{E}[S_T] = S_0 e^{\mu T}, \quad \text{Var}(S_T) = S_0^2 e^{2\mu T} (e^{\sigma^2 T} - 1), \quad (4)$$

and S_T is lognormal. The reproducible benchmark uses

$$S_0 = 100, \quad \mu = 0.05, \quad \sigma = 0.40, \quad T = 1. \quad (5)$$

The volatility is intentionally high enough to make the coarse-step positivity question visible while keeping a familiar annual horizon.

2.2 Non-affine stochastic volatility

Write $X_t = \log S_t$. The prescribed model is

$$dX_t = \left(\mu - \frac{1}{2}g(Y_t)^2\right)dt + g(Y_t)dW_t^{(1)}, \quad (6)$$

$$dY_t = \kappa(\theta - Y_t)dt + \xi\sqrt{1 + Y_t^2}dW_t^{(2)}, \quad (7)$$

$$g(y) = \sigma_{\min} + \frac{\sigma_{\max} - \sigma_{\min}}{1 + e^{-y}}, \quad d\langle W^{(1)}, W^{(2)} \rangle_t = \rho dt. \quad (8)$$

The benchmark parameters are

$$S_0 = 100, \quad Y_0 = 0, \quad \mu = 0.05, \quad \kappa = 2, \quad \theta = -0.2, \quad \xi = 0.6, \quad \rho = -0.7, \quad (9)$$

$$\sigma_{\min} = 0.10, \quad \sigma_{\max} = 0.50, \quad T = 1.$$

The logistic map guarantees $0.10 < g(y) < 0.50$. Its derivative is bounded, and $y \mapsto \sqrt{1 + y^2}$ is globally Lipschitz because the magnitude of its derivative is at most one. Thus the coefficients have the regularity and linear-growth structure needed for standard EM strong-convergence arguments; nevertheless, the finite reference used below remains a numerical benchmark, not a closed-form oracle.

3 Discretisation and mathematical analysis

3.1 Euler–Maruyama for GBM

On $t_n = nh$, $h = T/N$, let $\Delta W_n = W_{t_{n+1}} - W_{t_n} \sim \mathcal{N}(0, h)$. Freezing both coefficients at S_n and integrating over one step gives

$$S_{n+1}^{\text{EM}} = S_n^{\text{EM}} + \mu S_n^{\text{EM}} h + \sigma S_n^{\text{EM}} \Delta W_n = S_n^{\text{EM}} (1 + \mu h + \sigma \Delta W_n). \quad (10)$$

For globally Lipschitz drift and diffusion, EM has strong order 1/2 and weak order 1 for sufficiently smooth test functions [1, 2]. GBM coefficients are globally Lipschitz, so the standard result applies.

3.2 Milstein for GBM

The scalar Itô–Taylor expansion retains the iterated stochastic integral

$$\int_{t_n}^{t_{n+1}} \int_{t_n}^s dW_r dW_s = \frac{1}{2} [(\Delta W_n)^2 - h]. \quad (11)$$

For diffusion $b(s) = \sigma s$, $b'(s) = \sigma$, so the Milstein update is

$$S_{n+1}^{\text{Mil}} = S_n^{\text{Mil}} + \mu S_n^{\text{Mil}} h + \sigma S_n^{\text{Mil}} \Delta W_n + \frac{1}{2} \sigma^2 S_n^{\text{Mil}} [(\Delta W_n)^2 - h]. \quad (12)$$

Including (11) cancels the leading stochastic Taylor term omitted by EM, raising the scalar strong order to 1 while retaining weak order 1 [3].

3.3 Exact transition and pathwise coupling

For the same increment used in (10)–(12),

$$S_{n+1}^{\text{exact}} = S_n^{\text{exact}} \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)h + \sigma\Delta W_n\right]. \quad (13)$$

The exact, EM, and Milstein paths therefore share all ΔW_n . The terminal strong-error estimator is

$$\widehat{e}_s(h) = \frac{1}{M} \sum_{m=1}^M \left| S_{N,m}^{(h)} - S_{T,m}^{\text{exact}} \right|, \quad (14)$$

and a log–log least-squares slope estimates p in $\mathbb{E}|S_N - S_T| = O(h^p)$. Pairing is essential: independent exact paths would add $O(M^{-1/2})$ sampling variability unrelated to discretisation.

3.4 Weak error and an analytic bias oracle

For a test function φ , weak error is

$$e_w(h) = \left| \mathbb{E}\varphi(S_N^{(h)}) - \mathbb{E}\varphi(S_T) \right|. \quad (15)$$

For $\varphi(s) = s$, both EM and Milstein step multipliers have conditional mean $1 + \mu h$. Hence

$$\mathbb{E}[S_N^{\text{EM}}] = \mathbb{E}[S_N^{\text{Mil}}] = S_0(1 + \mu h)^N. \quad (16)$$

Using $N = T/h$ and $\log(1 + \mu h) = \mu h - \frac{1}{2}\mu^2 h^2 + O(h^3)$,

$$S_0(1 + \mu h)^{T/h} = S_0 \exp\left(\mu T - \frac{1}{2}\mu^2 T h + O(h^2)\right), \quad (17)$$

$$e_w(h) = \frac{1}{2} S_0 e^{\mu T} \mu^2 T h + O(h^2). \quad (18)$$

Thus the mean bias has exact asymptotic order one. The code reports both this deterministic bias and a paired Monte Carlo confidence interval for $S_N - S_T$; the former prevents sampling noise from hiding the weak rate.

3.5 Positivity analysis

EM is positive for a step only when

$$1 + \mu h + \sigma\sqrt{h}Z_n > 0, \quad (19)$$

so the one-step failure probability is

$$p_{\text{neg,EM}}(h) = \Phi\left(-\frac{1 + \mu h}{\sigma\sqrt{h}}\right) > 0. \quad (20)$$

For Milstein, the multiplier is the quadratic

$$q(z) = \frac{1}{2}\sigma^2 h z^2 + \sigma\sqrt{h}z + 1 + \mu h - \frac{1}{2}\sigma^2 h. \quad (21)$$

Its minimum is

$$q_{\min} = \frac{1}{2} + \mu h - \frac{1}{2}\sigma^2 h. \quad (22)$$

For (5) and $0 < h \leq 1$, this is positive, explaining why no Milstein positivity violation is observed here. Equation (22) also shows that this is parameter- and step-dependent, not unconditional positivity preservation. Exact/log-price simulation remains positive by construction.

3.6 Correlated EM for the non-affine model

Generate independent $Z_{1,n}, Z_{2,n} \sim \mathcal{N}(0, 1)$ and set

$$\Delta W_n^{(1)} = \sqrt{h} Z_{1,n}, \quad (23)$$

$$\Delta W_n^{(2)} = \sqrt{h} \left(\rho Z_{1,n} + \sqrt{1 - \rho^2} Z_{2,n} \right). \quad (24)$$

The EM update is

$$X_{n+1} = X_n + \left(\mu - \frac{1}{2} g(Y_n)^2 \right) h + g(Y_n) \Delta W_n^{(1)}, \quad (25)$$

$$Y_{n+1} = Y_n + \kappa(\theta - Y_n)h + \xi \sqrt{1 + Y_n^2} \Delta W_n^{(2)}, \quad (26)$$

$$S_{n+1} = e^{X_{n+1}} > 0. \quad (27)$$

The logistic function is evaluated with two algebraically equivalent branches to avoid overflow when $|Y_n|$ is large.

A scalar Milstein formula is *not* applied to (6)–(7). Written in two independent Brownian drivers, the diffusion vector fields have nonzero cross derivatives through $g(Y)$ and $\sqrt{1 + Y^2}$. A full multidimensional Milstein discretisation would generally require cross terms and simulated iterated stochastic integrals. Omitting them while claiming strong order one would be mathematically unjustified.

3.7 Nested-grid numerical reference

Let $h_{\text{ref}} = T/N_{\text{ref}}$. For a coarse step $h = Kh_{\text{ref}}$, the coupling is

$$\Delta W_{n,\text{coarse}}^{(i)} = \sum_{j=0}^{K-1} \Delta W_{Kn+j,\text{ref}}^{(i)}, \quad i = 1, 2. \quad (28)$$

Therefore coarse and fine solutions represent the same Brownian path. We use $N_{\text{ref}} = 2048$ for the main experiment and independently compare it with 4096 steps on 4000 fresh paths. For the call payoff $P(S_T) = (S_T - 100)^+$, paired differences estimate weak bias and substantially reduce variance. We report a weak trend only where its confidence interval is separated from zero.

3.8 Uncertainty quantification

For independent path-level observations $D_1, \dots, D_M$, every displayed Monte Carlo mean uses

$$\bar{D} \pm 1.96 \frac{s_D}{\sqrt{M}}, \quad s_D^2 = \frac{1}{M-1} \sum_{m=1}^M (D_m - \bar{D})^2. \quad (29)$$

The bands around fitted log–log lines are regression diagnostics, not repeated Monte Carlo uncertainty. Their slope standard errors quantify linear-fit uncertainty conditional on the chosen mesh range.

4 Implementation and reproducibility

4.1 Core algorithm

Algorithm 1 states the non-affine nested-grid calculation. GBM uses the same coupling pattern with the exact step (13).

Algorithm 1 Coupled non-affine EM convergence experiment

Input: Parameters (9), seed, path count M , fine count N_{ref} , coarse counts $\mathcal{N}$

Output: Strong errors, paired payoff biases, confidence intervals, diagnostics

- 1: **for** each memory-safe batch of paths **do**
 - 2: draw independent fine-grid arrays Z_1, Z_2
 - 3: construct correlated increments using (24)
 - 4: integrate (25)–(26) on the reference grid
 - 5: **for** $N \in \mathcal{N}$ **do**
 - 6: sum groups of N_{ref}/N increments using (28)
 - 7: integrate the coarse path and store paired terminal differences
 - 8: **end for**
 - 9: **end for**
 - 10: estimate means and confidence intervals using (29)
 - 11: validate increment variance/correlation, finiteness, and positivity
-

4.2 Single-file code structure

The requested submission has two primary source files. Accordingly, the code is organised as small, model-agnostic functions inside `financial_sde_project.py`, rather than split across modules.

Function group	Purpose
<code>gbm_terminal_coupled, run_gbm</code>	Exact/EM/Milstein coupling, strong and weak estimators, positivity and matched-cost results.
<code>logistic_vol,</code> <code>em_nonaffine_from_increments</code>	Stable model coefficients and swappable EM integration from supplied increments.
<code>aggregate_increments,</code> <code>run_nonaffine</code>	Nested Brownian coupling, reference comparison, call payoff and diagnostics.
<code>nonaffine_reference_refinement</code>	Independent 2048 $\rightarrow$ 4096 reference-resolution check.
<code>plot_*, validate</code>	Reproduce all five figures and execute ten PASS/FAIL checks.

Table 1: Code organisation. Integrators receive increments explicitly, so the random driver and numerical method can be tested independently.

4.3 Reproducible protocol

The default run uses seed 20260918, $M = 300,000$ GBM paths, $M = 20,000$ non-affine paths, batches of at most 1000 paths, and $N_{\text{ref}} = 2048$. The command

```
python financial_sde_project.py
```

regenerates `latex/generated/results.json`, `run_output.txt`, and Figures 1–5 with fixed names and at 220 DPI. No absolute paths are used. A reduced `-quick` mode is available only as a smoke test; all report numbers are from the default run. NumPy performs vectorised path operations [5], and Matplotlib produces the figures [6].

4.4 Oracle chain and automatic checks

Two independent GBM oracles are used: the pathwise exact transition and the closed-form terminal moments/distribution. The non-affine reference is checked by an independent finer grid. Ten automatic checks cover expected-order bands, the exact GBM mean, both increment variances, target correlation, finiteness, positivity, and a plausible non-affine strong slope. All ten pass in the archived output. The checks are deliberately broad enough to avoid passing only one random realisation, yet narrow enough to catch a wrong increment scaling, correlation formula, or scheme.

4.5 Declared cost measure

The cost measure is the number of time steps per path, equivalently one drift/diffusion coefficient evaluation per step in the implemented scalar GBM loop. The comparison excludes plotting, random-number setup, file I/O, compilation, and vector-allocation overhead. It also does not pretend that an EM and a Milstein step have identical floating-point operation counts: Milstein evaluates an extra correction. The reported ratio is therefore an algorithmic step-count comparison, not a wall-clock speedup claim.

5 Numerical experiments and results

5.1 GBM strong and weak convergence

Table 2 reports the asymptotic mesh range used for the fit. Figure 1 overlays the predicted slopes and 95% fit bands. The observed EM slope 0.4899 ± 0.0026 agrees with $1/2$; the Milstein slope 0.9099 ± 0.0167 approaches order one. The small shortfall from one is consistent with a finite pre-asymptotic mesh range rather than evidence for a different theoretical order. The analytic mean-bias slope 0.9982 ± 0.0004 is essentially one.

N	h	EM strong MAE	Milstein strong MAE	exact mean bias magnitude
4	0.250000	4.52887	0.460308	0.0325759
8	0.125000	3.28870	0.268833	0.0163567
16	0.062500	2.34715	0.149881	0.00819567
32	0.031250	1.67424	0.0800666	0.00410218
64	0.015625	1.18565	0.0414826	0.00205218
128	0.0078125	0.838329	0.0211342	0.00102636
256	0.00390625	0.594648	0.0106906	0.000513248

Table 2: GBM convergence with 300,000 coupled paths. Error decreases at the theoretically expected rates; the exact weak-bias column is common to EM and Milstein for the linear payoff $\varphi(s) = s$.

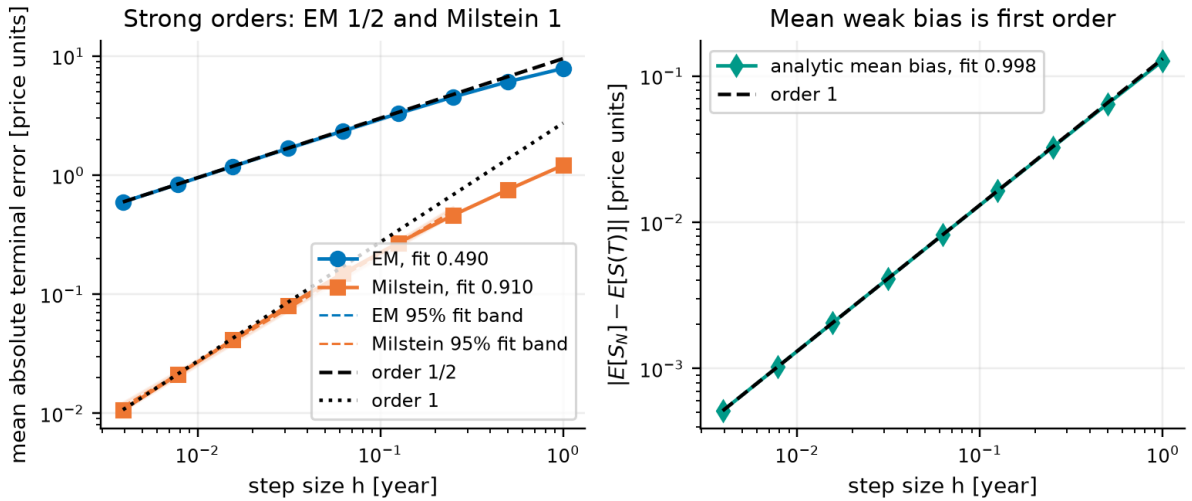


Figure 1: Coupled GBM convergence. Left: EM and Milstein terminal mean absolute errors follow the $h^{1/2}$ and h reference trends, respectively; fit bands make the slope uncertainty visible. Right: the analytically evaluable mean bias follows order one. This separates discretisation error from Monte Carlo noise and directly supports the claimed orders.

5.2 Matched-accuracy cost comparison

At the predeclared target $\mathbb{E}|S_N - S_T| \leq 0.85$, the coarsest tested EM grid that passes uses $N = 128$ and attains 0.83833; Milstein passes at $N = 2$ with 0.75433. Under the stated step-count measure, the work ratio is therefore

$$\frac{C_{\text{EM}}}{C_{\text{Mil}}} = \frac{128}{2} = 64. \quad (30)$$

This is a valid matched-accuracy comparison, but it is not a claim of a 64-fold wall-clock speedup. The negative result implicit in the comparison is also important: at very loose accuracy, EM's cheaper algebra could matter; that regime is not measured by this target.

5.3 Terminal distribution and moment validation

The exact formulas give

$$\mathbb{E}[S_T] = 105.127110, \text{quad Var}(S_T) = 1917.591686. \quad (31)$$

The exact-transition Monte Carlo sample gives 105.088894 and 1915.728882, respectively; the mean is within four Monte Carlo standard errors, satisfying the automatic oracle check. At $N = 64$, the 1%, 5%, 50%, 95%, and 99% quantiles are shown in Table 3.

Method	1%	5%	50%	95%	99%
Exact lognormal	38.2687	50.2610	97.0446	187.3749	246.0930
EM, $N = 64$	37.9146	50.0592	97.0464	187.5114	244.2812
Milstein, $N = 64$	38.3043	50.2674	96.9520	187.6476	245.1338

Table 3: GBM terminal quantiles in price units. Central quantiles agree closely; the tails reveal the remaining finite-step error more sensitively.

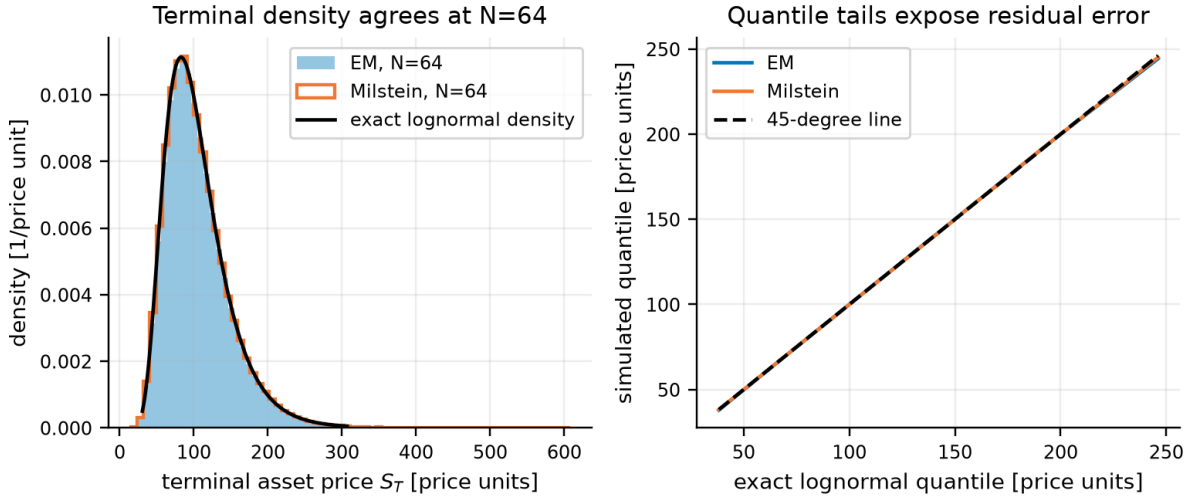


Figure 2: Terminal-law validation at $N = 64$. The density overlay checks the whole distribution rather than only two moments, while the quantile plot makes tail discrepancies visible. Milstein tracks the exact lognormal quantiles more closely than EM at the same step count.

5.4 Positivity stress test

At $h = 1$, 0.435% of EM paths encounter a non-positive step multiplier; at $h = 1/2$, the rate is 0.034%, and no violation is observed among 300,000 paths for $h \leq 1/4$. This is consistent with the rapid Gaussian-tail decay in (20). No Milstein violation occurs for the tested parameters, as predicted by (22).

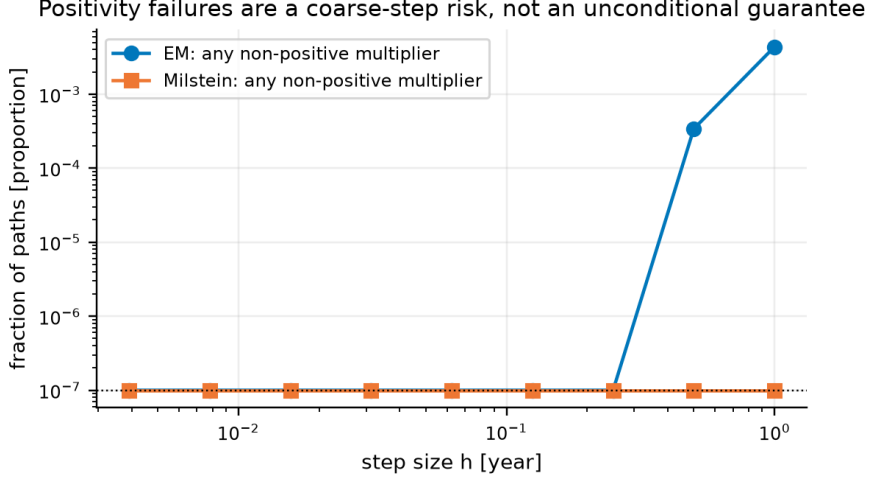


Figure 3: Observed fraction of paths with at least one non-positive update multiplier. EM fails on coarse grids even though the true GBM remains positive; zeros at finer grids mean “none in 300,000 paths”, not a mathematical guarantee. Milstein is positive here because its quadratic multiplier has a positive minimum for the tested h , not because Milstein is unconditionally positivity preserving.

5.5 Non-affine nested-grid convergence

Table 4 compares each coarse grid with the coupled 2048-step reference. Fitting the first four rows (the final row is reserved as a near-reference diagnostic) gives strong slope 0.5111 ± 0.0047 , closely matching the EM benchmark $1/2$. Every 95% confidence interval for the paired call-payoff difference contains zero. The correct conclusion is therefore that sampling uncertainty is already comparable to the estimated weak bias; the data do not support quoting a weak slope.

N	h	strong MAE	strong 95% CI	call bias (95% CI)
16	0.062500	0.886321	[0.875404, 0.897238]	0.004204 [−0.010095, 0.018504]
32	0.031250	0.627466	[0.619890, 0.635042]	0.003974 [−0.006056, 0.014005]
64	0.015625	0.441095	[0.435856, 0.446335]	0.003689 [−0.003286, 0.010664]
128	0.0078125	0.306001	[0.302342, 0.309660]	0.002141 [−0.002714, 0.006996]
256	0.00390625	0.210677	[0.208183, 0.213171]	0.002748 [−0.000568, 0.006064]

Table 4: Non-affine EM convergence against the coupled $N_{\text{ref}} = 2048$ numerical reference using 20,000 paths. Strong convergence is resolved, while the weak payoff bias is not statistically distinguishable from zero.

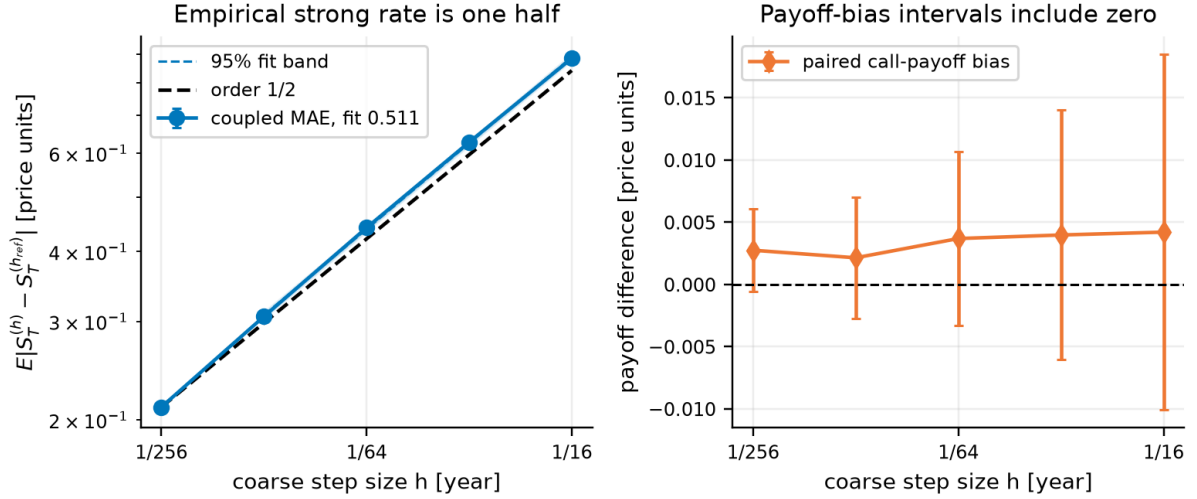


Figure 4: Nested-grid evidence for the non-affine model. Left: the pathwise terminal error follows an empirical half-order trend with narrow Monte Carlo intervals. Right: all payoff-bias confidence intervals meet zero, so claiming a weak order from these samples would over-interpret noise.

5.6 Increment, finiteness, positivity, and reference checks

The fine-grid increment diagnostics are

$$\frac{\widehat{\text{Var}}(\Delta W^{(1)})}{h} = 1.00526, \quad \frac{\widehat{\text{Var}}(\Delta W^{(2)})}{h} = 1.00346, \quad \widehat{\text{Corr}} = -0.70185, \quad (32)$$

against targets $(1, 1, -0.7)$. All 20,000 reference paths are finite and positive. Their estimated mean terminal price is 105.01815 with standard error 0.21433; the call value estimate is 14.48371 with standard error 0.15090. These are model outputs, not closed-form benchmarks.

On 4000 fresh coupled paths, refining the numerical reference from 2048 to 4096 steps changes the terminal price by mean absolute amount

$$0.057626 \quad \text{with 95\% CI } [0.056114, 0.059139]. \quad (33)$$

This is substantially smaller than the coarsest-grid strong errors and about 27% of the $N = 256$ error, so the reference is adequate for the observed trend, though not exact. A still finer reference would be required before reporting more terminal digits or resolving the weak payoff bias.

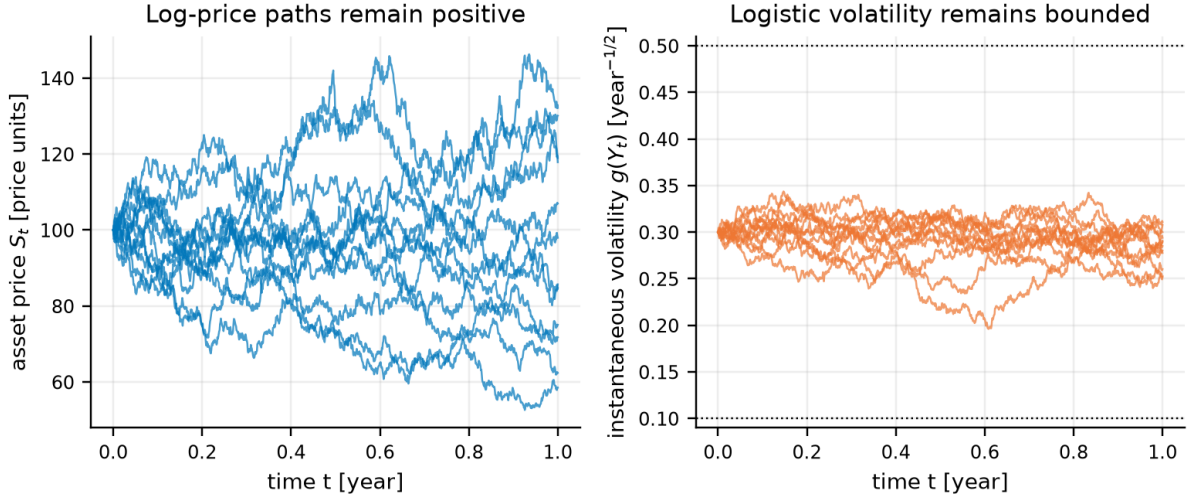


Figure 5: Representative non-affine paths. Integrating $X = \log S$ guarantees positive displayed prices after exponentiation (left), while the logistic map keeps instantaneous volatility strictly between 0.10 and $0.50 \text{ year}^{-1/2}$ (right). These structural properties are enforced by the model representation rather than inferred from a finite sample.

6 Limitations and robustness

1. **Numerical reference.** The non-affine error is relative to EM with 2048 steps. Equation (33) quantifies, but does not eliminate, reference bias.
2. **Weak convergence resolution.** Paired common random numbers reduce variance, yet the call-bias intervals still contain zero. The report therefore does not manufacture a weak slope. More paths, multilevel Monte Carlo, or a different payoff would be needed.
3. **Regression uncertainty.** Slope standard errors diagnose the chosen log-log fit. They are not theorem proofs and do not include every model-selection uncertainty, such as mesh-range choice.
4. **Cost definition.** Step count is portable but omits the extra arithmetic in Milstein and hardware effects. A wall-clock study should use repeated timings on the same machine and separately count random draws and coefficient/derivative evaluations.
5. **Model risk.** Numerical convergence does not validate GBM or the non-affine model as a description of market data. Calibration, risk-neutral drift, transaction costs, jumps, and empirical fit are outside this numerical-analysis project.
6. **Extreme paths.** The stable logistic evaluation and batched finiteness checks reduce numerical risk, but e^X can overflow under much longer horizons or more extreme parameters. Production code should retain log values and define an overflow policy.

7 Conclusions

- Exact GBM coupling recovered the expected strong orders: EM 0.4899 ± 0.0026 and Milstein 0.9099 ± 0.0167 ; the mean weak bias had order 0.9982 ± 0.0004 .
- At matched strong $\text{MAE} \leq 0.85$, Milstein required 2 steps versus 128 for EM under the declared step-count cost, while the report avoids converting that 64:1 ratio into an unsupported wall-clock claim.
- Distribution and moment checks agree with the exact lognormal oracle, and the coarse-grid stress test shows why price-space EM is not positivity-preserving.

- For the non-affine model, nested Brownian coupling produced empirical half-order strong convergence and passed variance, correlation, finiteness, positivity, and reference-refinement checks. Weak payoff bias remained unresolved at the available Monte Carlo precision.
- A natural extension is multilevel Monte Carlo, which would reuse the same nested coupling to reduce the cost of resolving weak payoffs [4]; a second extension is a correctly implemented multidimensional Milstein scheme with iterated stochastic integrals.

References

References

- [1] D. J. Higham, “An algorithmic introduction to numerical simulation of stochastic differential equations,” *SIAM Review*, vol. 43, no. 3, pp. 525–546, 2001. doi: [10.1137/S0036144500378302](https://doi.org/10.1137/S0036144500378302).
- [2] P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*. Berlin: Springer, 1992. doi: [10.1007/978-3-662-12616-5](https://doi.org/10.1007/978-3-662-12616-5).
- [3] G. N. Milstein, *Numerical Integration of Stochastic Differential Equations*. Dordrecht: Springer, 1995. doi: [10.1007/978-94-015-8455-5](https://doi.org/10.1007/978-94-015-8455-5).
- [4] M. B. Giles, “Multilevel Monte Carlo path simulation,” *Operations Research*, vol. 56, no. 3, pp. 607–617, 2008. doi: [10.1287/opre.1070.0496](https://doi.org/10.1287/opre.1070.0496).
- [5] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [6] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [7] MTH321 teaching team, *Project 1 Brief: Numerical Solution of ODE Initial Value Problems*, extension pathway 4, 2026, course material supplied with the project.
- [8] MTH321 teaching team, *Problem Pack: Five Suggested Directions for the ODE IVP Project*, “Financial SDE: GBM benchmark and non-affine volatility,” 2026, course material supplied with the project.

A Reproduction record and complete console output

A.1 Environment and commands

Install the declared dependencies in any isolated Python environment:

```
python -m pip install numpy matplotlib
```

From the repository root run

```
python financial_sde_project.py.
```

All paths are relative; the output directory is created automatically. The JSON file records Python, NumPy, Matplotlib, platform, seed, sample sizes, parameters, every table row, validation results, and wall time.

A.2 Archived program output

```
1 Financial SDE reproducibility run
2 seed=20260918
3 GBM fitted strong orders: EM=0.4899 +/- 0.0026, Milstein=0.9099 +/- 0.0167
4 GBM analytic weak-bias order=0.9982 +/- 0.0004
5 Matched strong-MAE cost: {"EM": {"mae": 0.8383291763996419, "n_steps": 128, "target_mae":
  0.85}, "Milstein": {"mae": 0.7543268709086007, "n_steps": 2, "target_mae": 0.85}, "
  work_ratio_em_over_milstein": 64.0}
6 GBM exact terminal mean: formula=105.127110, sample=105.088894
7 GBM exact terminal variance: formula=1917.591686, sample=1915.728882
8 Increment diagnostics: Var(dW1)/h=1.00526, Var(dW2)/h=1.00346, Corr=-0.70185
9 Non-affine empirical strong slope=0.5111 +/- 0.0047
10 Non-affine reference: {"finite_fraction": 1.0, "max_abs_y": 1.7965993064488124, "mean_call
  ": 14.48371142846275, "mean_s": 105.01815038210643, "min_s": 23.146331406720527, "
  n_steps": 2048, "positive_s_fraction": 1.0, "se_mean_call": 0.15090279205041998, "
  se_mean_s": 0.21433344042175662}
11 Reference refinement: {"ci_high": 0.059138620721274356, "ci_low": 0.05611432358184718, "
  coarse_reference_steps": 2048, "fine_reference_steps": 4096, "
  mean_abs_terminal_difference": 0.05762647215156077, "n_paths": 4000}
12 PASS: gbm_em_strong_order_in_expected_band
13 PASS: gbm_milstein_strong_order_in_expected_band
14 PASS: gbm_weak_order_in_expected_band
15 PASS: gbm_exact_mean_within_4_mc_se
16 PASS: brownian_variance_1_within_3_percent
17 PASS: brownian_variance_2_within_3_percent
18 PASS: brownian_correlation_within_0_02
19 PASS: nonaffine_all_paths_finite
20 PASS: nonaffine_all_prices_positive
21 PASS: nonaffine_empirical_strong_order_plausible
22 OVERALL: PASS
```

Listing 1: Complete output from the full reproducibility run.

B Complete Python source code

The following listing is the exact code used to generate every reported result and figure.

```
1 #!/usr/bin/env python3
2 """Reproducible experiments for the MTH321 Financial SDE project.
3
4 The script implements, rather than delegates, the numerical methods used in
5 the report: exact GBM coupling, Euler--Maruyama, scalar Milstein, and coupled
6 Euler--Maruyama for the two-factor non-affine stochastic-volatility model.
7
8 Run from the repository root:
9     python financial_sde_project.py
10
11 Dependencies: Python >= 3.10, NumPy, Matplotlib.
12 Outputs are written to latex/generated/ with stable file names. Use --quick
13 for a faster smoke test; the report numbers are produced by the default mode.
14 """
15
16 from __future__ import annotations
17
18 import argparse
19 import contextlib
20 import io
21 import json
22 import math
23 import platform
24 from statistics import NormalDist
25 import sys
26 import time
27 from dataclasses import asdict, dataclass
28 from pathlib import Path
29 from typing import Callable
30
31 import matplotlib
```

```

33 matplotlib.use("Agg")
34 import matplotlib.pyplot as plt
35 import numpy as np
36 from matplotlib.ticker import NullLocator
37
38
39 SEED = 20260918
40 ROOT = Path(__file__).resolve().parent
41 OUT = ROOT / "latex" / "generated"
42 CB = ["#0077BB", "#EE7733", "#009988", "#CC3311", "#33BBEE", "#000000"]
43 STD_NORMAL = NormalDist()
44
45 matplotlib.rcParams.update(
46     {
47         "font.family": "sans-serif",
48         "font.sans-serif": ["DejaVu Sans", "Arial"],
49         "font.size": 9,
50         "axes.titlesize": 10,
51         "axes.labelsize": 9,
52         "xtick.labelsize": 8,
53         "ytick.labelsize": 8,
54         "legend.fontsize": 8,
55         "figure.dpi": 160,
56         "savefig.dpi": 220,
57         "savefig.bbox": "tight",
58         "axes.spines.top": False,
59         "axes.spines.right": False,
60         "axes.grid": True,
61         "grid.alpha": 0.22,
62     }
63 )
64
65
66 @dataclass(frozen=True)
67 class GBMPParameters:
68     s0: float = 100.0
69     mu: float = 0.05
70     sigma: float = 0.40
71     maturity: float = 1.0
72
73
74 @dataclass(frozen=True)
75 class NonAffineParameters:
76     s0: float = 100.0
77     y0: float = 0.0
78     mu: float = 0.05
79     kappa: float = 2.0
80     theta: float = -0.2
81     xi: float = 0.6
82     rho: float = -0.7
83     sigma_min: float = 0.10
84     sigma_max: float = 0.50
85     maturity: float = 1.0
86     strike: float = 100.0
87
88
89 def ci95(x: np.ndarray) -> tuple[float, float, float]:
90     """Return sample mean and normal-approximation 95% confidence interval."""
91     x = np.asarray(x, dtype=float)
92     mean = float(np.mean(x))
93     half = 1.96 * float(np.std(x, ddof=1)) / math.sqrt(x.size)
94     return mean, mean - half, mean + half
95
96
97 def fitted_slope(h: np.ndarray, error: np.ndarray) -> tuple[float, float]:
98     """Log--log slope and its ordinary least-squares standard error."""
99     x, y = np.log(np.asarray(h)), np.log(np.asarray(error))
100     coef, cov = np.polyfit(x, y, 1, cov=True)
101     return float(coef[0]), float(math.sqrt(cov[0, 0]))
102
103
104 def add_loglog_fit_band(
105     ax: plt.Axes, h: np.ndarray, error: np.ndarray, color: str, label: str
106 ) -> None:

```

```

107     """Add an OLS log--log fit and a 95% pointwise regression band."""
108     lx, ly = np.log(np.asarray(h)), np.log(np.asarray(error))
109     coef, cov = np.polyfit(lx, ly, 1, cov=True)
110     gx = np.linspace(lx.min(), lx.max(), 100)
111     gy = coef[0] * gx + coef[1]
112     se = np.sqrt(cov[1, 1] + 2.0 * gx * cov[0, 1] + gx**2 * cov[0, 0])
113     ax.plot(np.exp(gx), np.exp(gy), "--", color=color, lw=1.0, label=label)
114     ax.fill_between(np.exp(gx), np.exp(gy - 1.96 * se), np.exp(gy + 1.96 * se), color=
        color, alpha=0.12)
115
116
117 def gbm_terminal_coupled(
118     p: GBMParameters, n_paths: int, n_steps: int, rng: np.random.Generator
119 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
120     """Couple exact GBM, EM, and Milstein with the same Brownian increments."""
121     h = p.maturity / n_steps
122     sqh = math.sqrt(h)
123     exact = np.full(n_paths, p.s0)
124     em = exact.copy()
125     mil = exact.copy()
126     ever_negative_em = np.zeros(n_paths, dtype=bool)
127     ever_negative_mil = np.zeros(n_paths, dtype=bool)
128     for _ in range(n_steps):
129         z = rng.standard_normal(n_paths)
130         dw = sqh * z
131         exact *= np.exp((p.mu - 0.5 * p.sigma**2) * h + p.sigma * dw)
132         em_factor = 1.0 + p.mu * h + p.sigma * dw
133         mil_factor = em_factor + 0.5 * p.sigma**2 * (dw**2 - h)
134         ever_negative_em |= em_factor <= 0.0
135         ever_negative_mil |= mil_factor <= 0.0
136         em *= em_factor
137         mil *= mil_factor
138     return exact, em, mil, ever_negative_em, ever_negative_mil
139
140
141 def run_gbm(p: GBMParameters, n_paths: int) -> dict:
142     steps = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256], dtype=int)
143     rows = []
144     terminal_for_distribution = None
145     for n in steps:
146         rng = np.random.default_rng(SEED + int(n))
147         exact, em, mil, neg_em, neg_mil = gbm_terminal_coupled(p, n_paths, int(n), rng)
148         em_abs = np.abs(em - exact)
149         mil_abs = np.abs(mil - exact)
150         em_diff = em - exact
151         mil_diff = mil - exact
152         em_ci = ci95(em_abs)
153         mil_ci = ci95(mil_abs)
154         weak_em_ci = ci95(em_diff)
155         weak_mil_ci = ci95(mil_diff)
156         analytic_mean_num = p.s0 * (1.0 + p.mu * p.maturity / n) ** n
157         exact_mean = p.s0 * math.exp(p.mu * p.maturity)
158         rows.append(
159             {
160                 "n_steps": int(n),
161                 "h": p.maturity / n,
162                 "strong_em": em_ci[0],
163                 "strong_em_ci_low": em_ci[1],
164                 "strong_em_ci_high": em_ci[2],
165                 "strong_milstein": mil_ci[0],
166                 "strong_milstein_ci_low": mil_ci[1],
167                 "strong_milstein_ci_high": mil_ci[2],
168                 "weak_em_paired": weak_em_ci[0],
169                 "weak_em_ci_low": weak_em_ci[1],
170                 "weak_em_ci_high": weak_em_ci[2],
171                 "weak_milstein_paired": weak_mil_ci[0],
172                 "weak_milstein_ci_low": weak_mil_ci[1],
173                 "weak_milstein_ci_high": weak_mil_ci[2],
174                 "weak_bias_exact": analytic_mean_num - exact_mean,
175                 "negative_path_em_rate": float(np.mean(neg_em)),
176                 "negative_path_milstein_rate": float(np.mean(neg_mil)),
177             }
178         )
179     if n == 64:

```

```

180         terminal_for_distribution = (exact.copy(), em.copy(), mil.copy())
181
182     h = np.array([r["h"] for r in rows])
183     strong_em = np.array([r["strong_em"] for r in rows])
184     strong_mil = np.array([r["strong_milstein"] for r in rows])
185     weak_exact = np.abs([r["weak_bias_exact"] for r in rows])
186     asymptotic = steps >= 4
187     slope_em, slope_em_se = fitted_slope(h[asymptotic], strong_em[asymptotic])
188     slope_mil, slope_mil_se = fitted_slope(h[asymptotic], strong_mil[asymptotic])
189     slope_weak, slope_weak_se = fitted_slope(h[asymptotic], weak_exact[asymptotic])
190
191     target = 0.85
192     matched = {}
193     for label, key in [("EM", "strong_em"), ("Milstein", "strong_milstein")]:
194         feasible = [r for r in rows if r[key] <= target]
195         selected = min(feasible, key=lambda r: r["n_steps"]) if feasible else None
196         matched[label] = (
197             {"target_mae": target, "n_steps": selected["n_steps"], "mae": selected[key]}
198             if selected
199             else {"target_mae": target, "n_steps": None, "mae": None}
200         )
201     if matched["EM"]["n_steps"] and matched["Milstein"]["n_steps"]:
202         matched["work_ratio_em_over_milstein"] = (
203             matched["EM"]["n_steps"] / matched["Milstein"]["n_steps"]
204         )
205
206     exact, em64, mil64 = terminal_for_distribution
207     exact_mean = p.s0 * math.exp(p.mu * p.maturity)
208     exact_var = p.s0**2 * math.exp(2 * p.mu * p.maturity) * (
209         math.exp(p.sigma**2 * p.maturity) - 1.0
210     )
211     distribution = {
212         "exact_mean_formula": exact_mean,
213         "exact_variance_formula": exact_var,
214         "exact_sample_mean": float(np.mean(exact)),
215         "exact_sample_variance": float(np.var(exact, ddof=1)),
216         "em_sample_mean_n64": float(np.mean(em64)),
217         "milstein_sample_mean_n64": float(np.mean(mil64)),
218         "quantiles": {},
219     }
220     probs = np.array([0.01, 0.05, 0.50, 0.95, 0.99])
221     exact_q = p.s0 * np.exp(
222         (p.mu - 0.5 * p.sigma**2) * p.maturity
223         + p.sigma * math.sqrt(p.maturity) * np.array([STD_NORMAL.inv_cdf(float(q)) for q
224             in probs])
225     )
226     for label, values in [("exact_formula", exact_q), ("EM_n64", np.quantile(em64, probs)),
227         ("Milstein_n64", np.quantile(mil64, probs))]:
228         distribution["quantiles"][label] = [float(v) for v in values]
229
230     plot_gbm_convergence(rows, slope_em, slope_mil, slope_weak)
231     plot_gbm_distribution(p, exact, em64, mil64)
232     plot_gbm_positivity(rows)
233     return {
234         "parameters": asdict(p),
235         "n_paths": n_paths,
236         "rows": rows,
237         "slopes": {
238             "strong_em": slope_em,
239             "strong_em_se": slope_em_se,
240             "strong_milstein": slope_mil,
241             "strong_milstein_se": slope_mil_se,
242             "weak_exact_bias": slope_weak,
243             "weak_exact_bias_se": slope_weak_se,
244         },
245         "matched_cost": matched,
246         "distribution": distribution,
247     }
248
249 def logistic_vol(y: np.ndarray, p: NonAffineParameters) -> np.ndarray:
250     """Bounded logistic volatility, evaluated without overflow."""
251     positive = y >= 0
252     q = np.empty_like(y, dtype=float)

```

```

252     q[positive] = 1.0 / (1.0 + np.exp(-y[positive]))
253     ey = np.exp(y[~positive])
254     q[~positive] = ey / (1.0 + ey)
255     return p.sigma_min + (p.sigma_max - p.sigma_min) * q
256
257
258 def em_nonaffine_from_increments(
259     p: NonAffineParameters, dw1: np.ndarray, dw2: np.ndarray, h: float
260 ) -> tuple[np.ndarray, np.ndarray]:
261     """Euler--Maruyama terminal state for supplied correlated increments."""
262     n_paths, n_steps = dw1.shape
263     x = np.full(n_paths, math.log(p.s0))
264     y = np.full(n_paths, p.y0)
265     for j in range(n_steps):
266         gy = logistic_vol(y, p)
267         x += (p.mu - 0.5 * gy**2) * h + gy * dw1[:, j]
268         y += p.kappa * (p.theta - y) * h + p.xi * np.sqrt(1.0 + y**2) * dw2[:, j]
269     return np.exp(x), y
270
271
272 def aggregate_increments(dw: np.ndarray, factor: int) -> np.ndarray:
273     n_paths, n_fine = dw.shape
274     if n_fine % factor:
275         raise ValueError("Fine grid must be divisible by the coarse-grid factor")
276     return dw.reshape(n_paths, n_fine // factor, factor).sum(axis=2)
277
278
279 def run_nonaffine(p: NonAffineParameters, n_paths: int, n_ref: int) -> dict:
280     coarse_steps = np.array([16, 32, 64, 128, 256], dtype=int)
281     if any(n_ref % n for n in coarse_steps):
282         raise ValueError("n_ref must be divisible by every coarse step count")
283     batch_size = min(1000, n_paths)
284     strong_samples = {int(n): [] for n in coarse_steps}
285     weak_samples = {int(n): [] for n in coarse_steps}
286     ref_payoffs = []
287     all_ref_s = []
288     all_ref_y = []
289     inc1 = []
290     inc2 = []
291     rng = np.random.default_rng(SEED + 5000)
292     h_ref = p.maturity / n_ref
293     for start in range(0, n_paths, batch_size):
294         b = min(batch_size, n_paths - start)
295         z1 = rng.standard_normal((b, n_ref))
296         z2 = rng.standard_normal((b, n_ref))
297         dw1 = math.sqrt(h_ref) * z1
298         dw2 = math.sqrt(h_ref) * (p.rho * z1 + math.sqrt(1.0 - p.rho**2) * z2)
299         inc1.append(dw1[:, : min(8, n_ref)].ravel())
300         inc2.append(dw2[:, : min(8, n_ref)].ravel())
301         s_ref, y_ref = em_nonaffine_from_increments(p, dw1, dw2, h_ref)
302         payoff_ref = np.maximum(s_ref - p.strike, 0.0)
303         all_ref_s.append(s_ref)
304         all_ref_y.append(y_ref)
305         ref_payoffs.append(payoff_ref)
306         for n in coarse_steps:
307             factor = n_ref // int(n)
308             c1 = aggregate_increments(dw1, factor)
309             c2 = aggregate_increments(dw2, factor)
310             s_coarse, _ = em_nonaffine_from_increments(p, c1, c2, p.maturity / n)
311             strong_samples[int(n)].append(np.abs(s_coarse - s_ref))
312             weak_samples[int(n)].append(np.maximum(s_coarse - p.strike, 0.0) - payoff_ref)
313
314     rows = []
315     for n in coarse_steps:
316         strong = np.concatenate(strong_samples[int(n)])
317         weak = np.concatenate(weak_samples[int(n)])
318         sc = ci95(strong)
319         wc = ci95(weak)
320         rows.append(
321             {
322                 "n_steps": int(n),
323                 "h": p.maturity / n,
324                 "strong_mae_vs_ref": sc[0],
325                 "strong_ci_low": sc[1],

```

```

326         "strong_ci_high": sc[2],
327         "call_bias_vs_ref": wc[0],
328         "call_bias_ci_low": wc[1],
329         "call_bias_ci_high": wc[2],
330     }
331 )
332 h = np.array([r["h"] for r in rows])
333 strong = np.array([r["strong_mae_vs_ref"] for r in rows])
334 slope, slope_se = fitted_slope(h[:-1], strong[:-1])
335
336 a = np.concatenate(inc1)
337 b = np.concatenate(inc2)
338 increment_diagnostics = {
339     "fine_h": h_ref,
340     "var_dw1_over_h": float(np.var(a, ddof=1) / h_ref),
341     "var_dw2_over_h": float(np.var(b, ddof=1) / h_ref),
342     "sample_correlation": float(np.corrcoef(a, b)[0, 1]),
343     "target_correlation": p.rho,
344 }
345 ref_s = np.concatenate(all_ref_s)
346 ref_y = np.concatenate(all_ref_y)
347 payoff = np.concatenate(ref_payoffs)
348 reference = {
349     "n_steps": n_ref,
350     "mean_s": float(np.mean(ref_s)),
351     "se_mean_s": float(np.std(ref_s, ddof=1) / math.sqrt(ref_s.size)),
352     "mean_call": float(np.mean(payoff)),
353     "se_mean_call": float(np.std(payoff, ddof=1) / math.sqrt(payoff.size)),
354     "min_s": float(np.min(ref_s)),
355     "max_abs_y": float(np.max(np.abs(ref_y))),
356     "finite_fraction": float(np.mean(np.isfinite(ref_s) & np.isfinite(ref_y))),
357     "positive_s_fraction": float(np.mean(ref_s > 0.0)),
358 }
359 refinement = nonaffine_reference_refinement(p, max(3000, n_paths // 5), n_ref)
360 plot_nonaffine_convergence(rows, slope)
361 plot_nonaffine_paths(p)
362 return {
363     "parameters": asdict(p),
364     "n_paths": n_paths,
365     "reference": reference,
366     "reference_refinement": refinement,
367     "increment_diagnostics": increment_diagnostics,
368     "rows": rows,
369     "empirical_strong_slope_excluding_fine": slope,
370     "empirical_strong_slope_se": slope_se,
371 }
372
373
374 def nonaffine_reference_refinement(
375     p: NonAffineParameters, n_paths: int, n_ref: int
376 ) -> dict:
377     """Compare h_ref with h_ref/2 on a fresh, coupled validation sample."""
378     n_fine = 2 * n_ref
379     rng = np.random.default_rng(SEED + 9000)
380     batch_size = min(500, n_paths)
381     differences = []
382     for start in range(0, n_paths, batch_size):
383         b = min(batch_size, n_paths - start)
384         z1 = rng.standard_normal((b, n_fine))
385         z2 = rng.standard_normal((b, n_fine))
386         hf = p.maturity / n_fine
387         d1f = math.sqrt(hf) * z1
388         d2f = math.sqrt(hf) * (p.rho * z1 + math.sqrt(1.0 - p.rho**2) * z2)
389         sf, _ = em_nonaffine_from_increments(p, d1f, d2f, hf)
390         d1c = aggregate_increments(d1f, 2)
391         d2c = aggregate_increments(d2f, 2)
392         sc, _ = em_nonaffine_from_increments(p, d1c, d2c, 2 * hf)
393         differences.append(np.abs(sc - sf))
394     diff = np.concatenate(differences)
395     est = ci95(diff)
396     return {
397         "n_paths": n_paths,
398         "coarse_reference_steps": n_ref,
399         "fine_reference_steps": n_fine,

```

```

400         "mean_abs_terminal_difference": est[0],
401         "ci_low": est[1],
402         "ci_high": est[2],
403     }
404
405
406 def plot_gbm_convergence(rows: list[dict], slope_em: float, slope_mil: float, slope_weak:
float) -> None:
407     h = np.array([r["h"] for r in rows])
408     em = np.array([r["strong_em"] for r in rows])
409     mil = np.array([r["strong_milstein"] for r in rows])
410     weak = np.abs([r["weak_bias_exact"] for r in rows])
411     fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.15))
412     ax = axes[0]
413     ax.loglog(h, em, "o-", color=CB[0], label=f"EM, fit {slope_em:.3f}")
414     ax.loglog(h, mil, "s-", color=CB[1], label=f"Milstein, fit {slope_mil:.3f}")
415     mask = h <= 0.25
416     add_loglog_fit_band(ax, h[mask], em[mask], CB[0], "EM 95% fit band")
417     add_loglog_fit_band(ax, h[mask], mil[mask], CB[1], "Milstein 95% fit band")
418     href = np.array([h.min(), h.max()])
419     ax.loglog(href, em[-1] * (href / h[-1]) ** 0.5, "--", color=CB[5], label="order 1/2")
420     ax.loglog(href, mil[-1] * (href / h[-1]), ":", color=CB[5], label="order 1")
421     ax.set_xlabel("step size h [year]")
422     ax.set_ylabel("mean absolute terminal error [price units]")
423     ax.set_title("Strong orders: EM 1/2 and Milstein 1")
424     ax.legend()
425     ax = axes[1]
426     ax.loglog(h, weak, "d-", color=CB[2], label=f"analytic mean bias, fit {slope_weak:.3f}")
427     ax.loglog(href, weak[-1] * (href / h[-1]), "--", color=CB[5], label="order 1")
428     ax.set_xlabel("step size h [year]")
429     ax.set_ylabel(r"$|E[S_N]-E[S(T)]|$ [price units]")
430     ax.set_title("Mean weak bias is first order")
431     ax.legend()
432     fig.tight_layout()
433     fig.savefig(OUT / "fig01_gbm_convergence.png")
434     plt.close(fig)
435
436
437 def plot_gbm_distribution(p: GBMParameters, exact: np.ndarray, em: np.ndarray, mil: np.
ndarray) -> None:
438     fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.15))
439     lo, hi = np.quantile(exact, [0.002, 0.998])
440     x = np.linspace(lo, hi, 400)
441     shape = p.sigma * math.sqrt(p.maturity)
442     scale = p.s0 * math.exp((p.mu - 0.5 * p.sigma**2) * p.maturity)
443     axes[0].hist(em, bins=70, density=True, alpha=0.42, color=CB[0], label="EM, N=64")
444     axes[0].hist(mil, bins=70, density=True, histtype="step", linewidth=1.4, color=CB[1],
label="Milstein, N=64")
445     density = np.exp(-0.5 * (np.log(x / scale) / shape) ** 2) / (x * shape * math.sqrt(2.0
* math.pi))
446     axes[0].plot(x, density, color=CB[5], lw=1.5, label="exact lognormal density")
447     axes[0].set_xlabel("terminal asset price $S_T$ [price units]")
448     axes[0].set_ylabel("density [1/price unit]")
449     axes[0].set_title("Terminal density agrees at N=64")
450     axes[0].legend()
451     probs = np.linspace(0.01, 0.99, 99)
452     q_formula = p.s0 * np.exp(
453         (p.mu - 0.5 * p.sigma**2) * p.maturity
454         + shape * np.array([STD_NORMAL.inv_cdf(float(q)) for q in probs])
455     )
456     axes[1].plot(q_formula, np.quantile(em, probs), color=CB[0], label="EM")
457     axes[1].plot(q_formula, np.quantile(mil, probs), color=CB[1], label="Milstein")
458     axes[1].plot([q_formula.min(), q_formula.max()], [q_formula.min(), q_formula.max()], "
--", color=CB[5], label="45-degree line")
459     axes[1].set_xlabel("exact lognormal quantile [price units]")
460     axes[1].set_ylabel("simulated quantile [price units]")
461     axes[1].set_title("Quantile tails expose residual error")
462     axes[1].legend()
463     fig.tight_layout()
464     fig.savefig(OUT / "fig02_gbm_distribution.png")
465     plt.close(fig)
466
467

```

```

468 def plot_gbm_positivity(rows: list[dict]) -> None:
469     h = np.array([r["h"] for r in rows])
470     em = np.array([r["negative_path_em_rate"] for r in rows])
471     mil = np.array([r["negative_path_milstein_rate"] for r in rows])
472     fig, ax = plt.subplots(figsize=(5.2, 3.2))
473     floor = 0.5 / max(1, rows[0].get("n_paths", 1))
474     ax.semilogy(h, np.maximum(em, 1e-7), "o-", color=CB[0], label="EM: any non-positive
475         multiplier")
476     ax.semilogy(h, np.maximum(mil, 1e-7), "s-", color=CB[1], label="Milstein: any non-
477         positive multiplier")
478     ax.axhline(1e-7, color=CB[5], ls=":", lw=0.8)
479     ax.set_xlabel("step size h [year]")
480     ax.set_xscale("log")
481     ax.set_ylabel("fraction of paths [proportion]")
482     ax.set_title("Positivity failures are a coarse-step risk, not an unconditional
483         guarantee")
484     ax.legend()
485     fig.tight_layout()
486     fig.savefig(OUT / "fig03_gbm_positivity.png")
487     plt.close(fig)
488
489 def plot_nonaffine_convergence(rows: list[dict], slope: float) -> None:
490     h = np.array([r["h"] for r in rows])
491     strong = np.array([r["strong_mae_vs_ref"] for r in rows])
492     slo = np.array([r["strong_ci_low"] for r in rows])
493     shi = np.array([r["strong_ci_high"] for r in rows])
494     bias = np.array([r["call_bias_vs_ref"] for r in rows])
495     blo = np.array([r["call_bias_ci_low"] for r in rows])
496     bhi = np.array([r["call_bias_ci_high"] for r in rows])
497     fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.15))
498     axes[0].errorbar(h, strong, yerr=[strong - slo, shi - strong], fmt="o-", capsize=2,
499         color=CB[0], label=f"coupled MAE, fit {slope:.3f}")
500     add_loglog_fit_band(axes[0], h[:-1], strong[:-1], CB[0], "95% fit band")
501     href = np.array([h.min(), h.max()])
502     axes[0].loglog(href, strong[-1] * (href / h[-1]) ** 0.5, "--", color=CB[5], label="
503         order 1/2")
504     axes[0].set_xscale("log")
505     axes[0].set_yscale("log")
506     axes[0].set_xlabel("coarse step size h [year]")
507     axes[0].set_ylabel("$E|S_T^{(h)} - S_T^{(h_{ref})}|$ [price units]")
508     axes[0].set_title("Empirical strong rate is one half")
509     axes[0].legend()
510     axes[1].errorbar(h, bias, yerr=[bias - blo, bhi - bias], fmt="d-", capsize=2, color=CB
511         [1], label="paired call-payoff bias")
512     axes[1].axhline(0.0, color=CB[5], ls="--", lw=1.0)
513     axes[1].set_xscale("log")
514     axes[1].set_xlabel("coarse step size h [year]")
515     axes[1].set_ylabel("payoff difference [price units]")
516     axes[1].set_title("Payoff-bias intervals include zero")
517     axes[1].legend()
518     tick_values = np.array([1.0 / 256.0, 1.0 / 64.0, 1.0 / 16.0])
519     for ax in axes:
520         ax.set_xticks(tick_values)
521         ax.set_xticklabels(["1/256", "1/64", "1/16"])
522         ax.xaxis.set_minor_locator(NullLocator())
523     fig.tight_layout()
524     fig.savefig(OUT / "fig04_nonaffine_convergence.png")
525     plt.close(fig)
526
527 def plot_nonaffine_paths(p: NonAffineParameters) -> None:
528     n_paths, n_steps = 12, 1024
529     h = p.maturity / n_steps
530     rng = np.random.default_rng(SEED + 12000)
531     x = np.full(n_paths, math.log(p.s0))
532     y = np.full(n_paths, p.y0)
533     s_hist = np.empty((n_steps + 1, n_paths))
534     g_hist = np.empty((n_steps + 1, n_paths))
535     s_hist[0] = np.exp(x)
536     g_hist[0] = logistic_vol(y, p)
537     for j in range(n_steps):
538         z1 = rng.standard_normal(n_paths)
539         z2 = rng.standard_normal(n_paths)

```

```

536     d1 = math.sqrt(h) * z1
537     d2 = math.sqrt(h) * (p.rho * z1 + math.sqrt(1.0 - p.rho**2) * z2)
538     gy = logistic_vol(y, p)
539     x += (p.mu - 0.5 * gy**2) * h + gy * d1
540     y += p.kappa * (p.theta - y) * h + p.xi * np.sqrt(1.0 + y**2) * d2
541     s_hist[j + 1] = np.exp(x)
542     g_hist[j + 1] = logistic_vol(y, p)
543 t = np.linspace(0.0, p.maturity, n_steps + 1)
544 fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.15))
545 for k in range(n_paths):
546     axes[0].plot(t, s_hist[:, k], lw=0.8, alpha=0.72, color=CB[0])
547     axes[1].plot(t, g_hist[:, k], lw=0.8, alpha=0.72, color=CB[1])
548 axes[0].set_xlabel("time t [year]")
549 axes[0].set_ylabel("asset price $S_t$ [price units]")
550 axes[0].set_title("Log-price paths remain positive")
551 axes[1].set_xlabel("time t [year]")
552 axes[1].set_ylabel("instantaneous volatility $g(Y_t)$ [year$^{-1/2}$]")
553 axes[1].set_ylim(p.sigma_min - 0.01, p.sigma_max + 0.01)
554 axes[1].axhline(p.sigma_min, color=CB[5], ls=":", lw=0.8)
555 axes[1].axhline(p.sigma_max, color=CB[5], ls=":", lw=0.8)
556 axes[1].set_title("Logistic volatility remains bounded")
557 fig.tight_layout()
558 fig.savefig(OUT / "fig05_nonaffine_paths.png")
559 plt.close(fig)
560
561
562 def validate(results: dict) -> dict:
563     gbm = results["gbm"]
564     non = results["nonaffine"]
565     checks = {
566         "gbm_em_strong_order_in_expected_band": 0.40 <= gbm["slopes"]["strong_em"] <=
567             0.65,
568         "gbm_milstein_strong_order_in_expected_band": 0.85 <= gbm["slopes"]["
569             strong_milstein"] <= 1.15,
570         "gbm_weak_order_in_expected_band": 0.90 <= gbm["slopes"]["weak_exact_bias"] <=
571             1.10,
572         "gbm_exact_mean_within_4_mc_se": abs(gbm["distribution"]["exact_sample_mean"] -
573             gbm["distribution"]["exact_mean_formula"])
574         <= 4.0 * math.sqrt(gbm["distribution"]["exact_variance_formula"] / gbm["n_paths"])
575         ,
576         "brownian_variance_1_within_3_percent": abs(non["increment_diagnostics"]["
577             var_dw1_over_h"] - 1.0) < 0.03,
578         "brownian_variance_2_within_3_percent": abs(non["increment_diagnostics"]["
579             var_dw2_over_h"] - 1.0) < 0.03,
580         "brownian_correlation_within_0_02": abs(non["increment_diagnostics"]["
581             sample_correlation"] - non["increment_diagnostics"]["target_correlation"]) <
582             0.02,
583         "nonaffine_all_paths_finite": non["reference"]["finite_fraction"] == 1.0,
584         "nonaffine_all_prices_positive": non["reference"]["positive_s_fraction"] == 1.0,
585         "nonaffine_empirical_strong_order_plausible": 0.35 <= non["
586             empirical_strong_slope_excluding_finest"] <= 0.75,
587     }
588     return {"checks": checks, "all_pass": bool(all(checks.values()))}
589
590
591 def print_summary(results: dict) -> None:
592     g = results["gbm"]
593     n = results["nonaffine"]
594     print("Financial SDE reproducibility run")
595     print(f"seed={SEED}")
596     print("GBM fitted strong orders: ")
597     f"EM={g['slopes']['strong_em']:.4f} +/- {g['slopes']['strong_em_se']:.4f}, "
598     f"Milstein={g['slopes']['strong_milstein']:.4f} +/- {g['slopes']['
599         strong_milstein_se']:.4f}"
600     print(f"GBM analytic weak-bias order={g['slopes']['weak_exact_bias']:.4f} +/- {g['
601         slopes']['weak_exact_bias_se']:.4f}")
602     print("Matched strong-MAE cost: " + json.dumps(g["matched_cost"], sort_keys=True))
603     print(f"GBM exact terminal mean: formula={g['distribution']['exact_mean_formula']:.6f
604         }, "
605         f"sample={g['distribution']['exact_sample_mean']:.6f}")
606     print(f"GBM exact terminal variance: formula={g['distribution']['
607         exact_variance_formula']:.6f}, "
608         f"sample={g['distribution']['exact_sample_variance']:.6f}")
609     d = n["increment_diagnostics"]

```

```

596 print(f"Increment diagnostics: Var(dw1)/h={d['var_dw1_over_h']:.5f}, "
597       f"Var(dw2)/h={d['var_dw2_over_h']:.5f}, Corr={d['sample_correlation']:.5f}")
598 print(f"Non-affine empirical strong slope={n['empirical_strong_slope_excluding_finetest']:.4f} "
599       f"+/- {n['empirical_strong_slope_se']:.4f}")
600 print("Non-affine reference: " + json.dumps(n["reference"], sort_keys=True))
601 print("Reference refinement: " + json.dumps(n["reference_refinement"], sort_keys=True))
602
603 for name, passed in results["validation"]["checks"].items():
604     print(f"{'PASS' if passed else 'FAIL'}: {name}")
605 print(f"OVERALL: {'PASS' if results['validation']['all_pass'] else 'FAIL'}")
606
607 def main() -> int:
608     parser = argparse.ArgumentParser(description=__doc__)
609     parser.add_argument("--quick", action="store_true", help="run a reduced smoke test")
610     args = parser.parse_args()
611     OUT.mkdir(parents=True, exist_ok=True)
612     gbm_paths = 40_000 if args.quick else 300_000
613     non_paths = 3_000 if args.quick else 20_000
614     n_ref = 512 if args.quick else 2048
615     start = time.perf_counter()
616     results = {
617         "metadata": {
618             "seed": SEED,
619             "quick_mode": args.quick,
620             "python": sys.version.split()[0],
621             "platform": platform.platform(),
622             "numpy": np.__version__,
623             "matplotlib": matplotlib.__version__,
624             "cost_measure": "time steps (one drift/diffusion coefficient evaluation per path-step); plotting, I/O, and setup excluded",
625         },
626         "gbm": run_gbm(GBMParameters(), gbm_paths),
627         "nonaffine": run_nonaffine(NonAffineParameters(), non_paths, n_ref),
628     }
629     results["validation"] = validate(results)
630     results["metadata"]["wall_time_seconds"] = time.perf_counter() - start
631     buffer = io.StringIO()
632     with contextlib.redirect_stdout(buffer):
633         print_summary(results)
634     output_text = buffer.getvalue()
635     print(output_text, end="")
636     (OUT / "run_output.txt").write_text(output_text, encoding="utf-8")
637     (OUT / "results.json").write_text(json.dumps(results, indent=2, sort_keys=True),
638                                       encoding="utf-8")
639     return 0 if results["validation"]["all_pass"] else 2
640
641 if __name__ == "__main__":
642     raise SystemExit(main())

```

Listing 2: Complete source: financial_sde_project.py.

C AI Transparency Log

C.1 Tools used

Tool	Version/date	Primary use
OpenAI Codex	18 September 2026	Requirements mapping, derivation drafting, Python implementation, experiment execution, result interpretation, and LaTeX assembly.
NumPy / Matplotlib	Versions recorded in <code>results.json</code>	Vectorised Monte Carlo computation and deterministic figure generation.

Table 5: AI and software tools. Team members must replace or supplement this record if they make later changes with other tools.

C.2 Significant AI-assisted claim audits

Item	Prompt/claim/assumptions	Risk and possible falsifier
1	Derive and test the GBM strong orders of EM and Milstein under pathwise coupling.	A wrong increment scale or uncoupled oracle can create a false slope. Falsifiers: exact-transition errors fail to decrease or fitted slopes leave the declared acceptance bands.
2	Use a nested EM reference for the non-affine model and describe the strong rate as empirical.	Treating a discretised reference as exact biases the claim. Falsifier: the 2048-to-4096 reference difference is comparable to the coarse-grid error or refinement ceases to reduce error.
3	Compare EM and Milstein cost at matched accuracy.	Counting only steps can overstate a speed advantage because a Milstein step has extra algebra. Falsifier: repeated wall-clock or operation-count measurements reverse the comparison at the same error target.

C.3 Independent verification and decisions

Item	Independent check and evidence location	Final decision/modification/limitation
1	Exact GBM transition, exact moments, analytic mean bias, Tables 2–3, and automatic order checks.	Accepted the strong/weak claims; reported fitted standard errors and separated analytic weak bias from Monte Carlo estimates.
2	Fresh coupled 2048-to-4096 refinement, Eq. (33), plus increment variance/correlation checks.	Called the result “empirical”; did not describe the reference as exact and did not claim a weak payoff slope.
3	Same strong-MAE threshold for both schemes and explicit cost exclusions in Section 4.	Reported a 64:1 step-count ratio only; explicitly withheld a wall-clock speedup claim.

C.4 Intellectual ownership declaration

Before submission, every member must complete and sign the table below. Each signer affirms that they can derive the equations, explain and modify the team-owned code, identify trusted library interfaces, and reproduce the validation without AI assistance during the viva.

Name	Signature	Team-owned artefact/contribution	Trusted dependency/interface	depen-
<Name 1>				
<Name 2>				
<Name 3>				
<Name 4>				
<Name 5>				

D Individual Contribution Statements

The following forms are intentionally not pre-filled: contribution claims must be written by the corresponding student and remain traceable to the team's Git history. Copy the appropriate concrete evidence (function, equation, figure, test, or report section) rather than writing "worked on the report."

Individual Contribution Statement — Project 1 (Financial SDE)

Name: <Name 1>

Role: <Project Manager>

The specific things I did this project:

1. <Name a file/function/equation/figure and the exact work done.>
2. <Name a validation or review action and its evidence location.>
3. <Name a report section or experiment and the decision made.>

The hardest conceptual obstacle and how I overcame it:

<Specific mathematical or algorithmic difficulty.>

One AI-assisted item I used and what I changed:

<Cross-reference AI Transparency Log item 1, 2, or 3.>

What I would do differently / what I still need to understand:

<Honest reflection.>

Signature: _____ **Date:** _____

Individual Contribution Statement — Project 1 (Financial SDE)**Name:** <Name 2>**Role:** <Mathematical Theory>**The specific things I did this project:**

1. <Name a file/function/equation/figure and the exact work done.>
2. <Name a validation or review action and its evidence location.>
3. <Name a report section or experiment and the decision made.>

The hardest conceptual obstacle and how I overcame it:

<Specific mathematical or algorithmic difficulty.>

One AI-assisted item I used and what I changed:

<Cross-reference AI Transparency Log item 1, 2, or 3.>

What I would do differently / what I still need to understand:

<Honest reflection.>

Signature: _____ **Date:** _____

Individual Contribution Statement — Project 1 (Financial SDE)

Name: <Name 3>

Role: <Algorithm Implementation>

The specific things I did this project:

1. <Name a file/function/equation/figure and the exact work done.>
2. <Name a validation or review action and its evidence location.>
3. <Name a report section or experiment and the decision made.>

The hardest conceptual obstacle and how I overcame it:

<Specific mathematical or algorithmic difficulty.>

One AI-assisted item I used and what I changed:

<Cross-reference AI Transparency Log item 1, 2, or 3.>

What I would do differently / what I still need to understand:

<Honest reflection.>

Signature: _____ **Date:** _____

Individual Contribution Statement — Project 1 (Financial SDE)

Name: <Name 4>

Role: <Visualization & Report>

The specific things I did this project:

1. <Name a file/function/equation/figure and the exact work done.>
2. <Name a validation or review action and its evidence location.>
3. <Name a report section or experiment and the decision made.>

The hardest conceptual obstacle and how I overcame it:

<Specific mathematical or algorithmic difficulty.>

One AI-assisted item I used and what I changed:

<Cross-reference AI Transparency Log item 1, 2, or 3.>

What I would do differently / what I still need to understand:

<Honest reflection.>

Signature: _____ **Date:** _____

Individual Contribution Statement — Project 1 (Financial SDE)

Name: <Name 5>

Role: <Testing & Validation>

The specific things I did this project:

1. <Name a file/function/equation/figure and the exact work done.>
2. <Name a validation or review action and its evidence location.>
3. <Name a report section or experiment and the decision made.>

The hardest conceptual obstacle and how I overcame it:

<Specific mathematical or algorithmic difficulty.>

One AI-assisted item I used and what I changed:

<Cross-reference AI Transparency Log item 1, 2, or 3.>

What I would do differently / what I still need to understand:

<Honest reflection.>

Signature: _____ **Date:** _____

E Submission evidence matrix

Course requirement	Evidence in this submission
SDE-specific methods and theory	Sections 2–3: exact GBM, EM, Milstein, strong/weak definitions, positivity, and coupled non-affine EM.
Independent oracle	Exact transition, exact moments/distribution, analytic mean bias, plus non-affine reference refinement.
Observed order for at least two methods	Table 2 and Figure 1, with slopes and fit uncertainty.
Monte Carlo uncertainty	Path-level 95% confidence intervals in Table 4 and explicit formula (29).
Matched-accuracy cost	Section 5: common MAE target 0.85, declared step-count cost, exclusions, and limitation.
Distribution and sanity checks	Figure 2, Table 3, moment oracle, increment variance/correlation, finiteness, positivity.
Non-affine nested validation	Equation (28), Table 4, Figure 4, and 2048-to-4096 refinement.
Reproducible code and figures	One command regenerates five fixed-name, 220-DPI figures, JSON, and console output; fixed seed; no absolute paths.
Interpretive visualisations	Every figure has units, a colorblind-safe palette, and a caption stating both the evidence and its significance.
Scientific limitations	Section 6 separates numerical convergence, reference error, sampling uncertainty, cost, and model risk.
AI log and ICS	Appendices C and D, with audit/falsifier records and five member-specific forms.